

CS 1101 Unit 4

Programming Assignment

Assignment Response

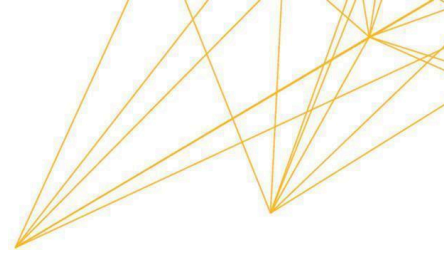
1. Hypotenuse Calculator

You work as a software developer in a company that creates custom software solutions for various clients. Your company has been approached by an educational client who wants to develop a function that calculates the length of the hypotenuse of a right triangle given the lengths of the other two legs as arguments. Your manager has instructed you to use incremental development to create the necessary function and document each stage of the development process. After completing the final stage of development, you have to test the function with different arguments and record the outputs in your Learning Journal.

Tasks

Include all of the following in your submission:

- An explanation of each stage of development, including code and any test input and output.
- The output of `hypotenuse(3,4)`.
- The output of two additional calls to `hypotenuse` with different arguments.



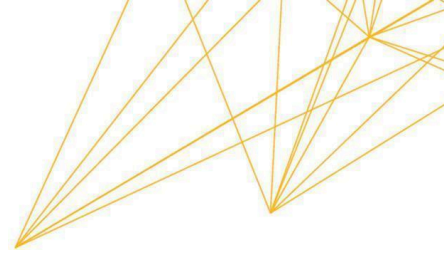
Stage 1: Function Outline and Syntactic Verification

To begin the development of the hypotenuse function, I followed the process where “*the first step is to consider what a function should look like in Python. In other words, what are the inputs (parameters) and what is the output (return value)?*” (Downey, 2015, p. 52). For this task, the inputs are the two legs of the triangle (a and b), and the output is the length of the hypotenuse.

```
hypotenuseCalculator.py U X
Programming Assignment Unit 4 > hypotenuseCalculator.py
1
2 # Part 1: Hypotenuse Calculator
3
4 def hypotenuse(a, b):
5     print("a is", a) # Scaffolding
6     print("b is", b) # Scaffolding
7     return 0.0
8
9 hypotenuse(3, 4)
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main !1 ?1 | python3 hypotenuseCalculator.py
a is 3
b is 4
```

Technical Explanation: At this stage, the function “*is syntactically correct, and it runs, which means that you can test it before you make it more complicated*” (Downey, 2015, p. 53). The print statements here are a form of scaffolding, used to verify that the arguments are correctly passed to the parameters.



Stage 2: Using Temporary Variables for Intermediate Math

In the second stage, I began adding the logic for the Pythagorean theorem. To keep the process manageable, *“a reasonable next step is to find the [squares of the legs]. The next version stores those values in temporary variables and prints them”* (Downey, 2015, p. 53).

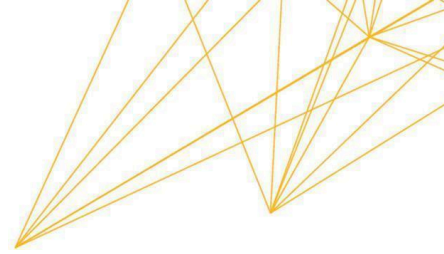
```
hypotenuseCalculator.py U X
Programming Assignment Unit 4 > hypotenuseCalculator.py
1
2 # Part 1: Hypotenuse Calculator
3
4 def hypotenuse(a, b):
5     aSquared = a ** 2
6     bSquared = b ** 2
7     print("a squared is", aSquared)
8     print("b squared is", bSquared)
9     return 0.0
10
11 hypotenuse(3, 4)
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main !1 ?1 | python3 hypotenuseCalculator.py
a squared is 9
b squared is 16
```

Technical Explanation: I used temporary variables (aSquared and bSquared) because, as Downey notes, it is helpful to *“use variables to hold intermediate values so you can display and check them”* (2015, p. 54). This ensures the exponentiation logic is correct before proceeding.

Stage 3: Calculating the Sum of Squares

Following the formula $a^2 + b^2$, I added a line to calculate the sum.



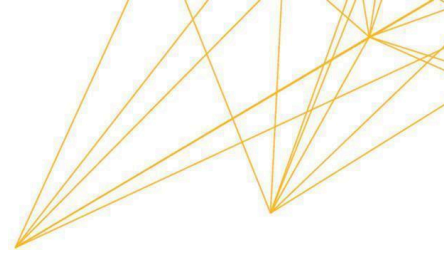
```
hypotenuseCalculator.py U X
Programming Assignment Unit 4 > hypotenuseCalculator.py
1
2 # Part 1: Hypotenuse Calculator
3
4 def hypotenuse(a, b):
5     aSquared = a ** 2
6     bSquared = b ** 2
7     hSquared = aSquared + bSquared
8     print("sum squared is", hSquared)
9     return 0.0
10
11 hypotenuse(3, 4)
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main !1 ?1 | python3 hypotenuseCalculator.py
sum squared is 25
```

Technical Explanation: By running the program at this stage, I “*check the output (which should be 25)*” (Downey, 2015, p. 53). This incremental step confirms that the addition of the intermediate squares is functioning as intended.

Stage 4: Final Development and Removing Scaffolding

In the final stage, I imported the math module to use the square root function. After confirming the math was correct, I removed the debugging prints. As Downey explains, “*The print statements we wrote are useful for debugging, but once you get the function working, you should remove them. Code like that is called scaffolding because it is helpful for building the program but is not part of the final product*” (2015, p. 53).

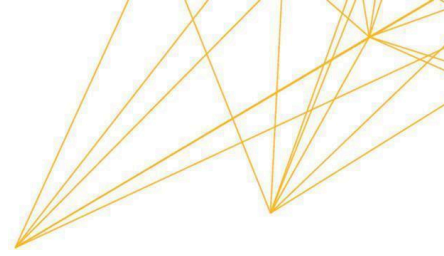


```
hypotenuseCalculator.py U X
Programming Assignment Unit 4 > hypotenuseCalculator.py
1  import math
2
3  # Part 1: Hypotenuse Calculator
4
5  def hypotenuse(a, b):
6      aSquared = a ** 2
7      bSquared = b ** 2
8      hSquared = aSquared + bSquared
9      result = math.sqrt(hSquared)
10     return result
11
12 # Final test calls
13 print("Call 1:", hypotenuse(3, 4))
14 print("Call 2:", hypotenuse(5, 12))
15 print("Call 3:", hypotenuse(8, 15))
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main !1 ?1 python3 hypotenuseCalculator.py
Call 1: 5.0
Call 2: 13.0
Call 3: 17.0
```

Function Call	Output
hypotenuse(3, 4)	5.0
hypotenuse(5, 12)	13.0
hypotenuse(8, 15)	17.0

Summary of the Process: The goal of this assignment was to use incremental development to *“avoid long debugging sessions by adding and testing only a small amount of code at a time”* (Downey, 2015, p. 52). By starting with a working program and making small changes, I ensured that *“if there is an error, [I] should have a good idea where it is”* (Downey, 2015, p. 54). The



final function is a fruitful function that correctly calculates the hypotenuse using the square root of the sum of the squares of the two legs.

```
import math

# Part 1: Hypotenuse Calculator

def hypotenuse(a, b):

    aSquared = a ** 2

    bSquared = b ** 2

    hSquared = aSquared + bSquared

    result = math.sqrt(hSquared)

    return result

# Final test calls

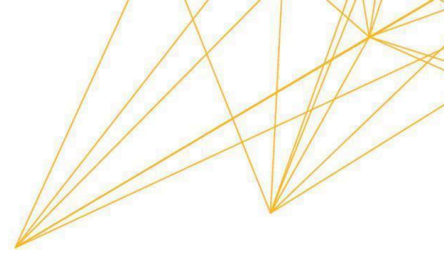
print("Call 1:", hypotenuse(3, 4))

print("Call 2:", hypotenuse(5, 12))

print("Call 3:", hypotenuse(8, 15))
```

2. Custom Function Portfolio

You are a software developer who wants to establish yourself as a skilled and versatile programmer. To achieve this, you have decided to create a work portfolio that showcases your

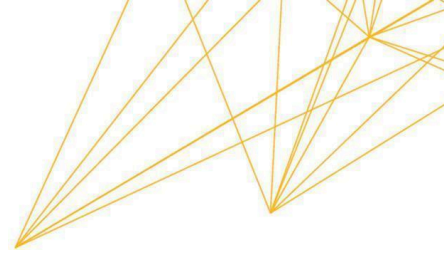


ability to develop custom software solutions. This portfolio will be your gateway to attract potential clients and establish yourself as a freelancer. As part of your portfolio, you plan to create your own function that does some useful computation using an incremental development approach that will demonstrate your programming skills and problem-solving abilities. You will document each stage of the development process, including the code and any test input and output in your Programming Assignment.

As part of my professional portfolio, I have developed a function to calculate Body Mass Index (BMI). BMI is a universally recognized metric used to categorize an individual's body mass relative to their height. The formula used is $BMI = \frac{weight}{height^2}$, where weight is in kilograms and height is in meters.

Stage 1: Initial Outline and Parameter Verification

The first step in my incremental development process was *“to consider what a [BMI] function should look like in Python. In other words, what are the inputs (parameters) and what is the output (return value)?”* (Downey, 2015, p. 52). I defined the function to accept weight and height.



```
customFunctionPortfolio.py U x
Programming Assignment Unit 4 > customFunctionPortfolio.py
1
2 # Part 2: Custom Function Portfolio
3
4 def calculate_bmi(weight, height):
5     print("Weight input:", weight) # Scaffolding
6     print("Height input:", height) # Scaffolding
7     return 0.0
8
9 calculate_bmi(70, 1.75)
```

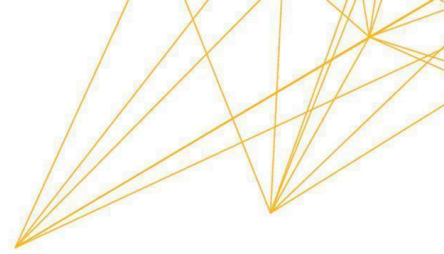
```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main !1 ?1 | python3 customFunctionPortfolio.py
Weight input: 70
Height input: 1.75
```

Technical Explanation: This ensures the function is “*syntactically correct, and it runs*”

(Downey, 2015, p. 53). By using scaffolding (the print statements), I verified that the function correctly receives numerical arguments before applying the BMI formula.

Stage 2: Calculating Height Squared

In this stage, I focused on the denominator of the formula $height^2$. I followed the advice to “*use variables to hold intermediate values so you can display and check them*” (Downey, 2015, p. 54).



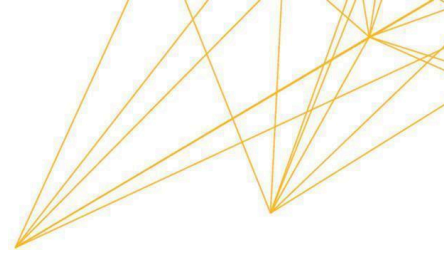
```
customFunctionPortfolio.py U X
Programming Assignment Unit 4 > customFunctionPortfolio.py
1
2 # Part 2: Custom Function Portfolio
3
4 def calculate_bmi(weight, height):
5     heightSquared = height ** 2 # Temporary variable
6     print("Height squared is:", heightSquared) # Scaffolding
7     return 0.0
8
9 calculate_bmi(70, 1.75)

~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main !1 ?1 python3 customFunctionPortfolio.py
Height squared is: 3.0625
```

Technical Explanation: I created a temporary variable called heightSquared. Checking this value independently is a *“reasonable next step”* (Downey, 2015, p. 53) because it ensures the exponentiation logic is accurate before it is used in the final division.

Stage 3: Final Calculation and Scaffolding Removal

The final stage involved completing the division and returning a “fruitful” result. Once I confirmed the math was correct, I removed the debugging code. As Downey notes, *“once you get the function working, you should remove them. Code like that is called scaffolding because it is helpful for building the program but is not part of the final product.”* (2015, p. 53).



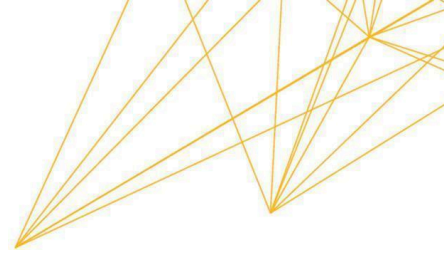
```
customFunctionPortfolio.py U x
Programming Assignment Unit 4 > customFunctionPortfolio.py
1
2 # Part 2: Custom Function Portfolio
3
4 def calculate_bmi(weight, height):
5     heightSquared = height ** 2
6     bmiResult = weight / heightSquared
7     return bmiResult
8
9 # Three calls with different arguments
10 print("BMI Test 1 (Normal):", calculate_bmi(70, 1.75))
11 print("BMI Test 2 (Underweight):", calculate_bmi(50, 1.80))
12 print("BMI Test 3 (Overweight):", calculate_bmi(100, 1.70))
13
14 calculate_bmi(70, 1.75)
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 4 | main | 1 ? | python3 customFunctionPortfolio.py
BMI Test 1 (Normal): 22.857142857142858
BMI Test 2 (Underweight): 15.432098765432098
BMI Test 3 (Overweight): 34.602076124567475
```

Function Call	Output	Category (Reference)
calculate_bmi(70, 1.75)	22.857...	Normal
calculate_bmi(50, 1.80)	15.432...	Underweight
calculate_bmi(100, 1.70)	34.602...	Obese

Technical Explanation

The final function successfully implements the BMI formula using Python’s arithmetic operators. By using an incremental development plan, I avoided “*long debugging sessions by adding and testing only a small amount of code at a time*” (Downey, 2015, p. 52). The use of temporary variables like heightSquared ensured that the order of operations was followed



correctly. The function now provides a precise floating-point value that can be used for health analysis. This process demonstrates how *“breaking a large program into smaller functions creates natural checkpoints for debugging”* (Downey, 2015, p. 59), ensuring that the return value is accurate and the logic is sound.

Part 2: Custom Function Portfolio

```
def calculate_bmi(weight, height):  
    heightSquared = height ** 2  
    bmiResult = weight / heightSquared  
    return bmiResult  
  
# Three calls with different arguments  
print("BMI Test 1 (Normal):", calculate_bmi(70, 1.75))  
print("BMI Test 2 (Underweight):", calculate_bmi(50, 1.80))  
print("BMI Test 3 (Overweight):", calculate_bmi(100, 1.70))  
  
calculate_bmi(70, 1.75)
```

References

Downey, A. (2015). Think Python: How to think like a computer scientist. Green Tree Press.

<https://greenteapress.com/thinkpython2/thinkpython2.pdf>